



NUST CHIP DESIGN CENTRE

RISC-V Assembly

Lab Manual # 14

Calling Convention in RISC-V Assembly

<u>Name</u>	<i><u>Muddassir Ali Siddiqui</u></i>
<u>Instructor</u>	<i><u>Sir Imran & Sir Umer</u></i>
<u>Date</u>	<i><u>19th Aug 2025</u></i>

1. In-Lab Tasks: (Write your lab task & screenshots here)

i. Questions:

Q: Ask yourself: why did we not save t1 before using it?

A: Because temporary registers donot need to be preserved.

Q: Do we need to save and restore all of these registers?

A: Only t0 register is saved and restored. While it did not require to save t1, t2, t3.

Q: Think to yourself, which piece of the calling convention is missing?

A: Here ra is required to store because function calls itself so it is necessary to return at correct address.

Q: Which callee saved registers do we need to save?

A: We must save and restore s0, s1, s2, s3 in the prologue and epilogue.

s0 for sum

s1 for rs

s2 for rd

s3 for offset

ii. Task 1:

```
[CC Violation]: (PC=0x0000004C) Setting of a saved register (s0) which has not been saved! editor.S:54 li s0, 1
[CC Violation]: (PC=0x00000054) Setting of a saved register (s0) which has not been saved! editor.S:57 mul s0, s0, a0
[CC Violation]: (PC=0x00000054) Setting of a saved register (s0) which has not been saved! editor.S:57 mul s0, s0, a0
[CC Violation]: (PC=0x00000054) Setting of a saved register (s0) which has not been saved! editor.S:57 mul s0, s0, a0
[CC Violation]: (PC=0x00000054) Setting of a saved register (s0) which has not been saved! editor.S:57 mul s0, s0, a0
[CC Violation]: (PC=0x00000054) Setting of a saved register (s0) which has not been saved! editor.S:57 mul s0, s0, a0
[CC Violation]: (PC=0x00000054) Setting of a saved register (s0) which has not been saved! editor.S:57 mul s0, s0, a0
[CC Violation]: (PC=0x00000054) Setting of a saved register (s0) which has not been saved! editor.S:57 mul s0, s0, a0
[CC Violation]: (PC=0x00000064) Save register s0 not correctly restored before return! Expected 0x00000000, Actual 0x00000080.
editor.S:65 jr ra
[CC Violation]: (PC=0x00000070) Setting of a saved register (s0) which has not been saved! editor.S:79 mv s0, a0 # Copy start of
array to saved register
[CC Violation]: (PC=0x00000074) Setting of a saved register (s1) which has not been saved! editor.S:80 mv s1, a1 # Copy length of
array to saved register
[CC Violation]: (PC=0x000000A4) Setting of a saved register (s0) which has not been saved! editor.S:117 addi s0, t1, 1

Found 12 warnings!

-----

[ERROR] An error has occurred!

Error:

'StoreError: You are attempting to edit the text of the program though the program is set to immutable at address 0x00000006!

ID:

'RunStart'!
```

1. Is next_test a function?

It is label inside a main.

2. What caused the errors in pow, and inc_arr that were reported by the Venus CC checker?

Update s0 without saved it.

Update s0, s1 without preserved and called the function without saving loop counter.

3. In RISC-V, we call functions by jumping to them and storing the return address in the ra register. Does calling convention apply to the jumps to the pow_loop or pow_end labels?

No, it is used for functions only not for jumps in the function.

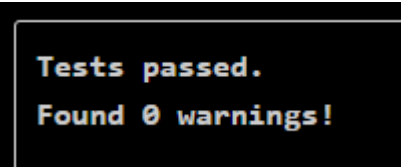
4. Why do we need to store ra in the prologue for inc_arr, but not in any other function?

Because it calls another function. So, we want to return back that's why it is necessary to save the ra.

5. Why wasn't the calling convention error in helper_fn reported by the CC checker? (Hint: it's mentioned above in the exercise instructions.)

It is not declared at .globl, so the checker doesn't analyze it.

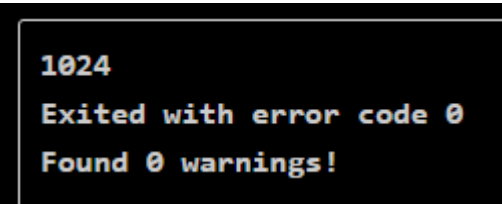
After the error solving the output is



```
Tests passed.  
Found 0 warnings!
```

iii. Task 2:

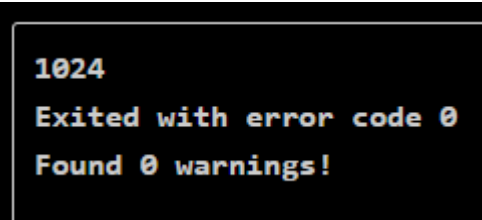
Only add ra store and load command



```
1024  
Exited with error code 0  
Found 0 warnings!
```

iv. Task 3:

Only add ra store and load command



```
1024  
Exited with error code 0  
Found 0 warnings!
```

2. Critical Analysis: (Write your critical analysis / conclusion here)

In this lab we need to save the return address and saved registers in memory to restore it back because we want to come back at the exact return address if we not do this the program behaves vulnerable.